Aalto University
School of Science

GenAI/LLMs and Big Data Platforms: Overview and Trends

Hong-Linh Truong
Department of Computer Science
linh.truong@aalto.fi

18.3.2026

Content is available under
CC BY-SA 4.0 unless otherwise stated

Learning objectives

- Understand the role of GenAI/LLMs as services for data platforms
- Understand the current trends in supporting GenAI and AI agents in state-of-the-art data platforms
- Able to support the development of GenAI/LLMs using techniques and methods studied from data platforms

What we have learned so far

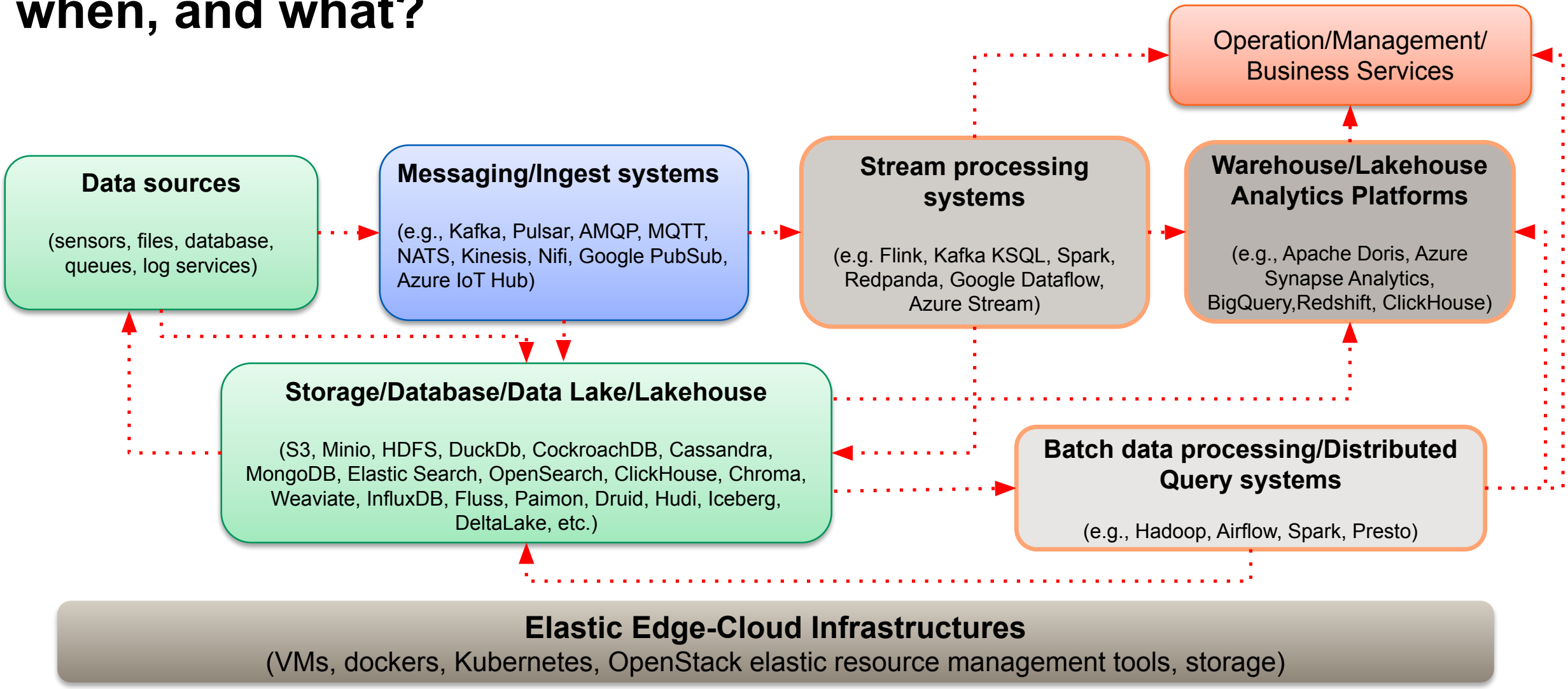
- Big data platforms are complex. We have
  - studied basic, fundamental techniques and models
  - practiced with existing platforms, and developed our own use cases/solutions
  - focused on design and testing
- Current focuses and requirements in modern data platforms
  - near real-time data and analytics
  - platform modernization with data mesh, lakehouse, design patterns like medallion architectures
  - composibility and interoperability of platform components and subsystems

GenAI/LLMs as a service

- Foundation Models, GenAI, Large Language Models (LLMs)
  - foundation models learn patterns and build their capabilities based on data
  - LLMs: foundation models trained with a lot of data, able to handle and generate natural languages and other contents (generative models)
  - GenAI uses generative models to analyze and generate text, code, images, data, etc.
- AI Agent software
  - acting on behalf of the user/customer to plan and complete tasks for the user (goal-oriented)
  - “intelligence” built around/powered by GenAI/LLMs
  - capable of autonomy, proactiveness, collaboration, environment sensing, memorization, and uncertainty handling
- Agentic System
  - include many AI agents and techniques/tools to coordinate the agents to achieve goals autonomously

GenAI/LLMs for Data Platforms *vs* Data Platforms for GenAI/LLMs

Find the role of GenAI/LLM in big data at large-scale: where, when, and what?



GenAI/LLMs & data platforms

- Exploring the benefits of GenAI/LLMs
  - for the development and use cases in big data platforms
- Integrating GenAI/LLMs services
  - for platform requirement analysis, design and development
- Providing GenAI/LLMs services as platform features
  - for data analysis, visualization, and transformation
- Building data platforms as a backbone for AI/ML/LLMs
  - LLMs requires strong support from big data techniques, models and platforms
- Leveraging the expertise in big data platforms for building GenAI/LLMs
  - data pipeline, data services, data transformation, and data quality/governance

Potentials of GenAI/LLMs: for what? saving cost? improving user experience? enabling low-code/no-code?

GenAI/LLMs as a solution for design and implementation of big data platforms

- Can GenAI/LLMs help for speeding up designs of big data platforms?
- Can GenAI/LLMs reduce programming tasks?
- Can GenAI/LLMs do the analytics of data by invoking suitable algorithms?
- etc.

Example of using GenAI for design understanding

Copy only 2 sentences in a question of Assignment 2, replaced “mybdp” with “platform”

tenants will design transformation pipelines which analyze the bronze data to
create silver data and the platform will execute these pipelines for the tenants. Design a schema for a
set of constraints for tenant service agreement that the platform will support the pipelines.

Answer from a free ChatGPT

1. Core Concept

The **Tenant Service Agreement (TSA)** defines limits and guarantees for pipeline execution:

- resource usage
- data access
- execution frequency
- latency guarantees
- concurrency
- reliability / retries
- cost limits
- security restrictions

The platform validates pipelines against these constraints before execution.

2. High-Level Schema

Example **logical schema**:

```
TenantServiceAgreement
│
├── tenant_id
├── pipeline_constraints
├── resource_limits
├── data_access_constraints
├── execution_constraints
├── reliability_constraints
├── cost_constraints
└── security_constraints
```

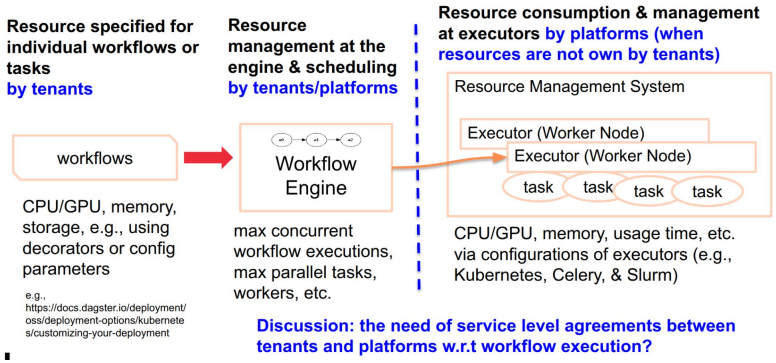
3. Example JSON Schema

A practical schema format could be **JSON/YAML**, because pipelines are often registered via API.

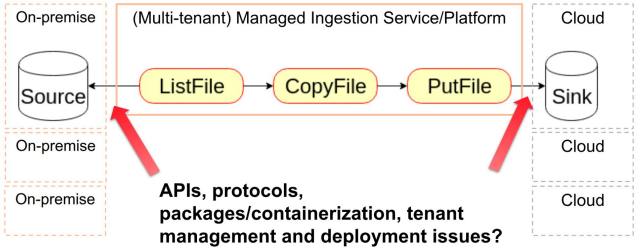
```JSON
{
  "tenant_id": "string",

  "pipeline_constraints": {
    "max_pipelines": "integer",
    "max_pipeline_dag_nodes": "integer",
    "allowed_languages": ["sql", "python", "spark"],
    "max_pipeline_runtime_minutes": "integer"
  },

  "resource_limits": {
    "max_cpu_cores": "integer",
    "max_memory_gb": "integer",
    "max_storage_gb": "integer",
    "max_parallel_jobs": "integer",
    "max_gpu": "integer"
  },

  "data_access_constraints": {
    "allowed_bronze_sources": ["dataset_id"],
    "allowed_silver_targets": ["dataset_id"],
    "max_input_size_gb": "integer",
    "max_output_size_gb": "integer"
  },
```

Recall: the concept we have learned

Scheduling and resource management (2)



Complex deployment and composition models (4)



Tools for the developer but also for platform functionality

- Code generation
  - Many cloud services (with close and open source LLM models)
  - Claude Code, Antigravity, Gemini, ChatGPT, Azure Mistral, together.ai
- Text-to-SQL for analytics
  - Vanna - https://vanna.ai/docs/
  - https://github.com/HKUSTDial/NL2SQL_Handbook
- LLMs for data management
  - https://github.com/weAIDB/awsome-data-llm

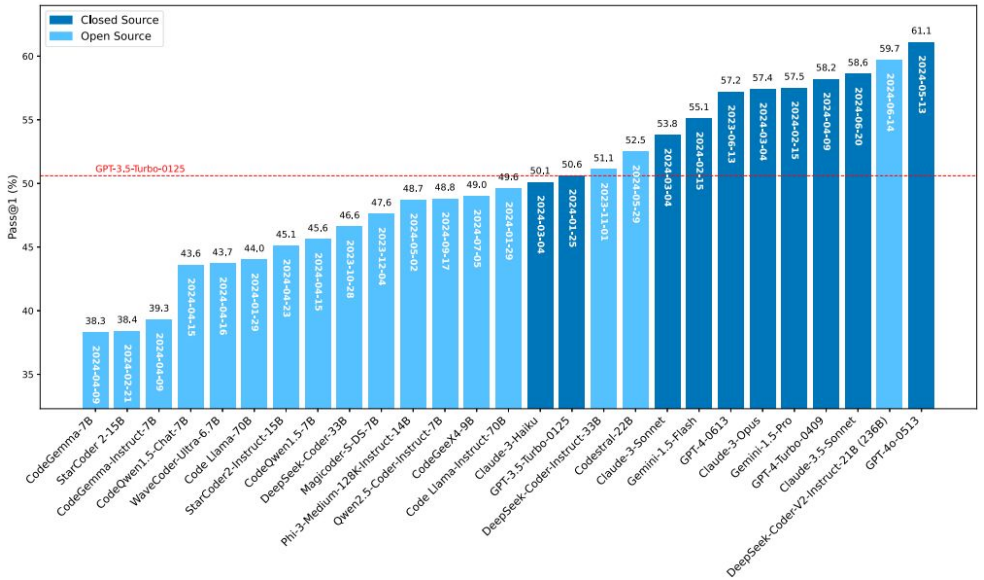


Fig. 19. The performance comparison of LLMs for code generation on the BigCodeBench [332] benchmark, measured by pass@1. For models with various sizes, we report only the largest size version of each model with a magnitude of billion (B) parameters.

Figure source: Jiang et al., A Survey on Large Language Models for Code Generation. ACM Trans. Softw. Eng. Methodol. 35, 2, Article 58 (February 2026), https://doi.org/10.1145/3747588

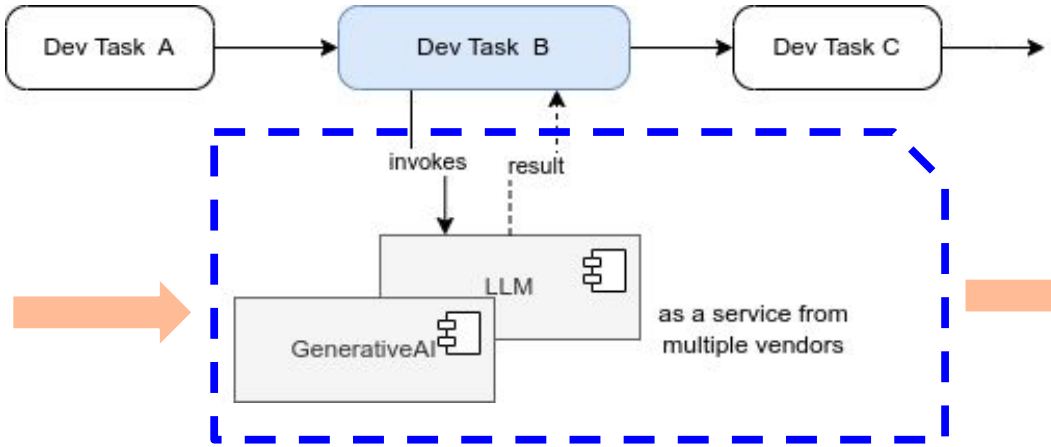
Understand the expected quality of results from GenAI/LLMs

Aspects/Views

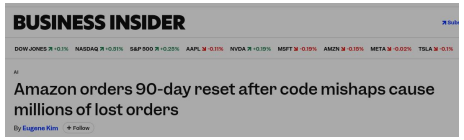
as a process
(e.g., inference process for an input)

as an artefact
(e.g., output)

as a service
(e.g., an ML service for a type of tasks)



trusted, reliable, bias/fair, secure, cost-effective results?



liability, regulation, risk, ethics, sustainability

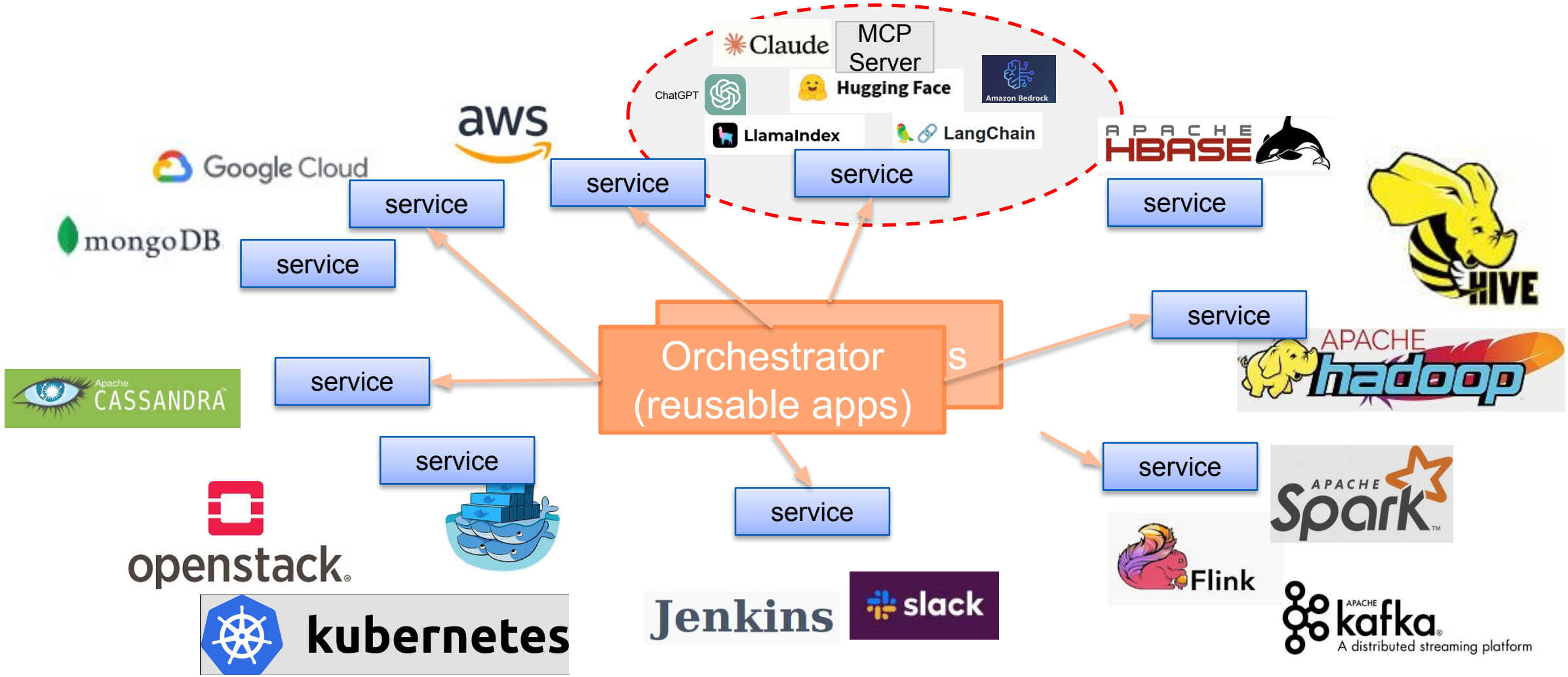
Must be considered and interpreted concretely in the view of data platforms!

Understand the need to coordinate GenAI/ LLMs and services for development tasks

- Composition of GenAI/LLMs with other development tools/knowledge sources
  - integrated with conventional highly accurate and relevant services
  - cloud pay-per-use and on-prem LLMs services
- Human-in-loop as workflow tasks
  - for uncertainty handling and quality feedback
- Adaptive coordination workflows
  - based on quality associated with individual LLM capabilities

Solution designs including GenAI/LLMs as service in a big data platform

Service orchestration in data platforms



Example of workflows supporting GenAI/LLM tasks

Airflow AI SDK

Features

- **LLM tasks with** `@task.llm` : Define tasks that call language models to process text
- **Agent tasks with** `@task.agent` : Orchestrate multi-step AI reasoning with custom tools
- **Automatic output parsing:** Use type hints to automatically parse and validate LLM outputs
- **Branching with** `@task.llm_branch` : Change DAG control flow based on LLM output
- **Model support:** All models in the Pydantic AI library (OpenAI, Anthropic, Gemini, etc.)
- **Embedding tasks with** `@task.embed` : Create vector embeddings from text

Source: https://github.com/astronomer/airflow-ai-sdk?tab=readme-ov-file

Dagster with OpenAI, Anthropic, etc.

```
import dagster as dg


@dg.asset(compute_kind="anthropic")

@dg.asset(compute_kind="OpenAI")
```

Source: https://docs.dagster.io/integrations/libraries

An increasing support, as data analytics workflows want to leverage diverse types of AI capabilities

Employing LLMs: service selection and integration problems

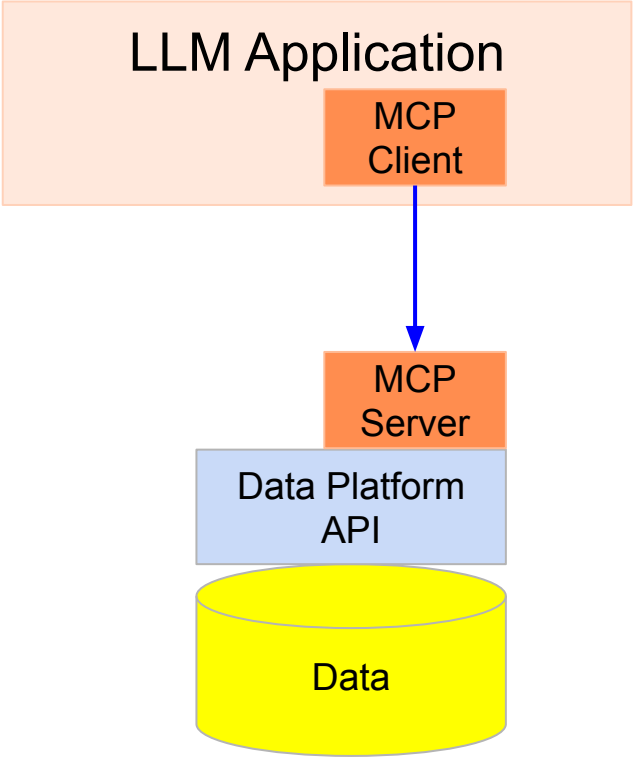
- Big LLMs
  - mostly from well-known providers
  - must send data to GenAI/LLMs services ⇒ complex constraints among the tenant, the platform and the external GenAI provider
- Small LLMs
  - can be hosted within a platform, can be dedicated for individual tenants
- Diversity of LLMs
  - functionality: generating analytics reports, code, sample of data
- Engineering challenges
  - scalability, data privacy/governance, and multi-tenant models

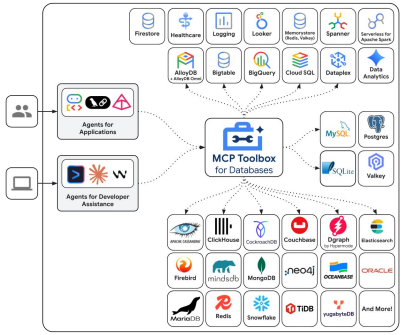
Ddata platforms requirements are different from other types of platforms

MCP for Data Services

- **MCP (Model Context Protocol)**
  - enable AI applications to interact with external tools, systems and services
  - making data from other sources for GenAI/LLMs
- **Why is MCP increasingly integrated with data platforms?**
  - LLMs/GenAI can query many types of data from databases/data storage → making valuable data for GenAI/LLMs to support applications

    ⇒ MCP as a protocol for exchanging data in data platforms that we shall consider
- **Key concerns**
  - performance: overhead and high latency
  - data access control, security: data may be shifted to external LLMs
  - management: create and delete data ⇒ dangerous operations





MCP Toolbox for Databases

Source: https://github.com/googleapis/genai-toolbox/tree/main

Simple examples: for analytics and development

Claude CLI + MCP

MCP Toolbox for Databases

tip_amount	FLOAT	
tolls_amount	FLOAT	
improvement_surcharge	FLOAT	
total_amount	FLOAT	
congestion_surcharge	FLOAT	

Table stats:
- **Rows:** 1,369,765
- **Size:** ~178 MB
- **Type:** TABLE (non-partitioned, non-clustered)
- **Created:** 2025-02-01

This is a NYC Yellow Taxi trip dataset with standard TLC (Taxi & Limousine Commission) schema fields.

❯ how many trips we have in the taxi dataset?

● **bigquery - execute_sql (MCP)**(sql: "SELECT COUNT(*) AS total_trips FROM `ssotesting-449013.taxitesting.taxi_trips`")
└ {
"total_trips": 1369765
}

● There are **1,369,765 trips** in the taxi_trips dataset.

❯ What is the average trip distance and total revenue?

Model: Default Sonnet 4.6 · Best for everyday tasks
Memory: project (CLAUDE.md)
Setting sources:

Claude

BigQuery

Sonnet 4.6 · Claude Pro
~/…/bigdataplatforms/tutorials/mcpbigdata/claude

Recent activity
No recent activity

❯ find bigquery data structure using mcp

● Let me explore the BigQuery data structure using the available MCP tools.

● **bigquery - list_dataset_ids (MCP)**
└ "taxitesting"

● **bigquery - list_table_ids (MCP)**(dataset: "taxitesting")
└ "taxi_trips"

● **bigquery - get_table_info (MCP)**(dataset: "taxitesting", table: "taxi_trips")
└ {
"Name": "",
"Location": "EU",
… +63 lines (ctrl+o to expand)

● Here's the BigQuery data structure found:

Project: ssotesting-449013

Dataset: taxitesting (Location: EU)

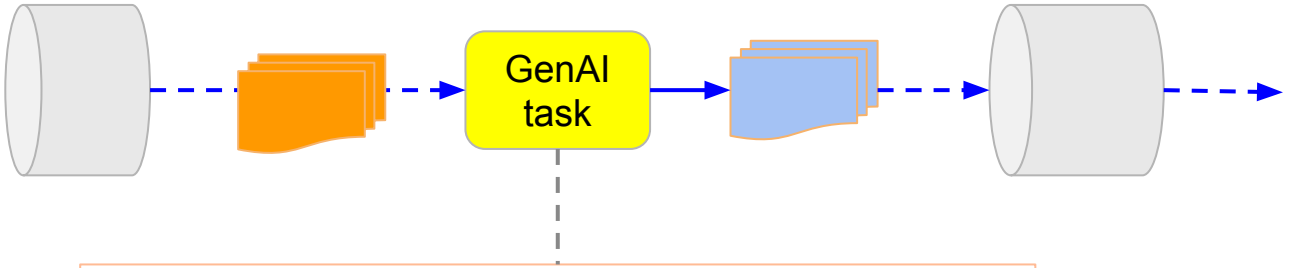
Table: taxi_trips

Column	Type	Notes
VendorID	INTEGER	
tpep_pickup_datetime	TIMESTAMP	

Some MPC plugins for data analytics:
Financial services:
https://github.com/anthropics/financial-services-plugins
Government data: https://github.com/datagouv/datagouv-mcp
Ecomerce: https://shopify.dev/docs/apps/build/storefront-mcp

GenAI supporting data ingestion and processing

Data ingestions and processing tasks



- **Calling GenAI**
  - process text, structured data
  - extract data
  - profile data
  - search and identify suitable data tables/fields

Example: Google data engineering agent

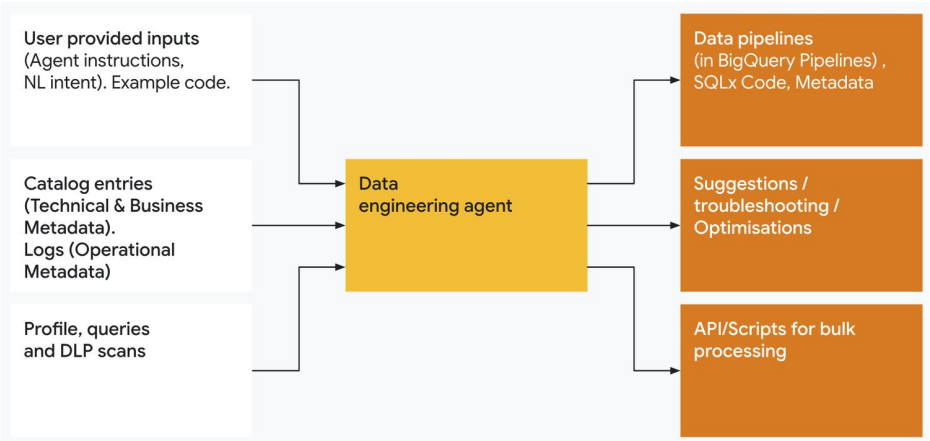


Figure source:
https://cloud.google.com/blog/products/data-analytics/a-closer-look-at-bigquery-data-engineering-agent

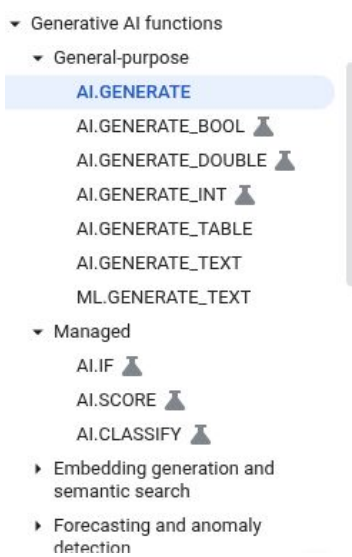
Reading:
https://www.vldb.org/2025/Workshops/VLDB-Workshops-2025/DEC/DEC25_5.pdf

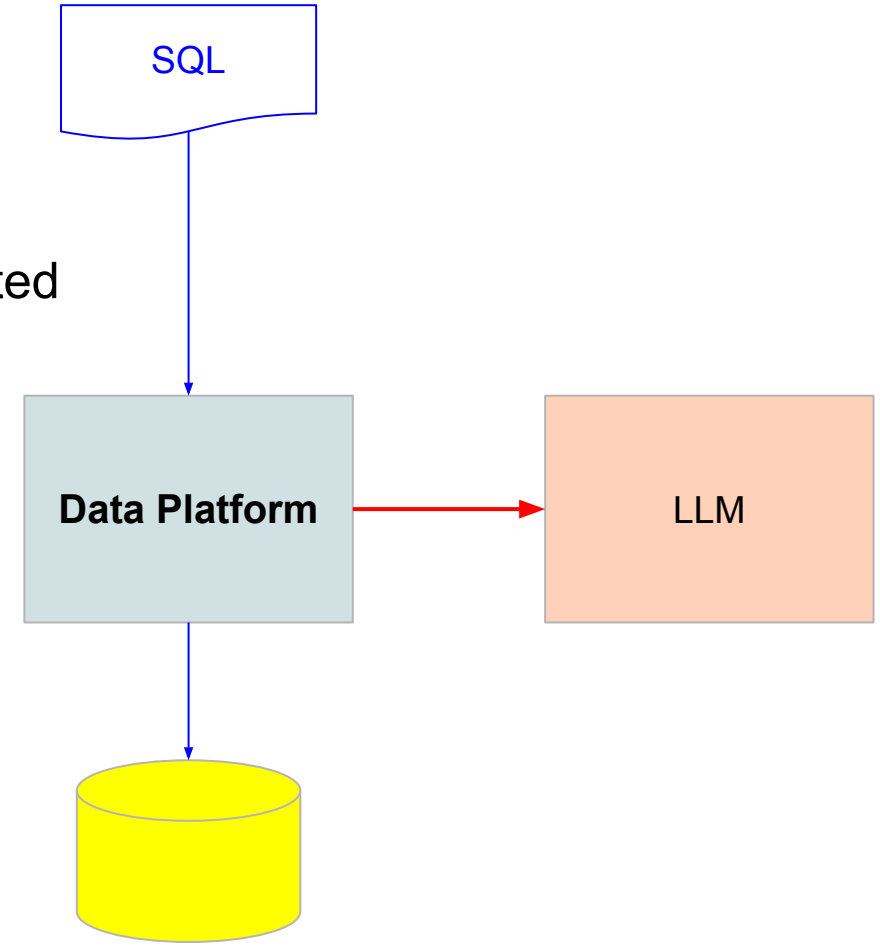
GenAI supporting data analytics

- Data platform services call LLMs
  - AI-generated report/texts for query engines
  - AI inferencing and scoring for analytics

⇒ coupling advanced AI functions with data analytics in a platform

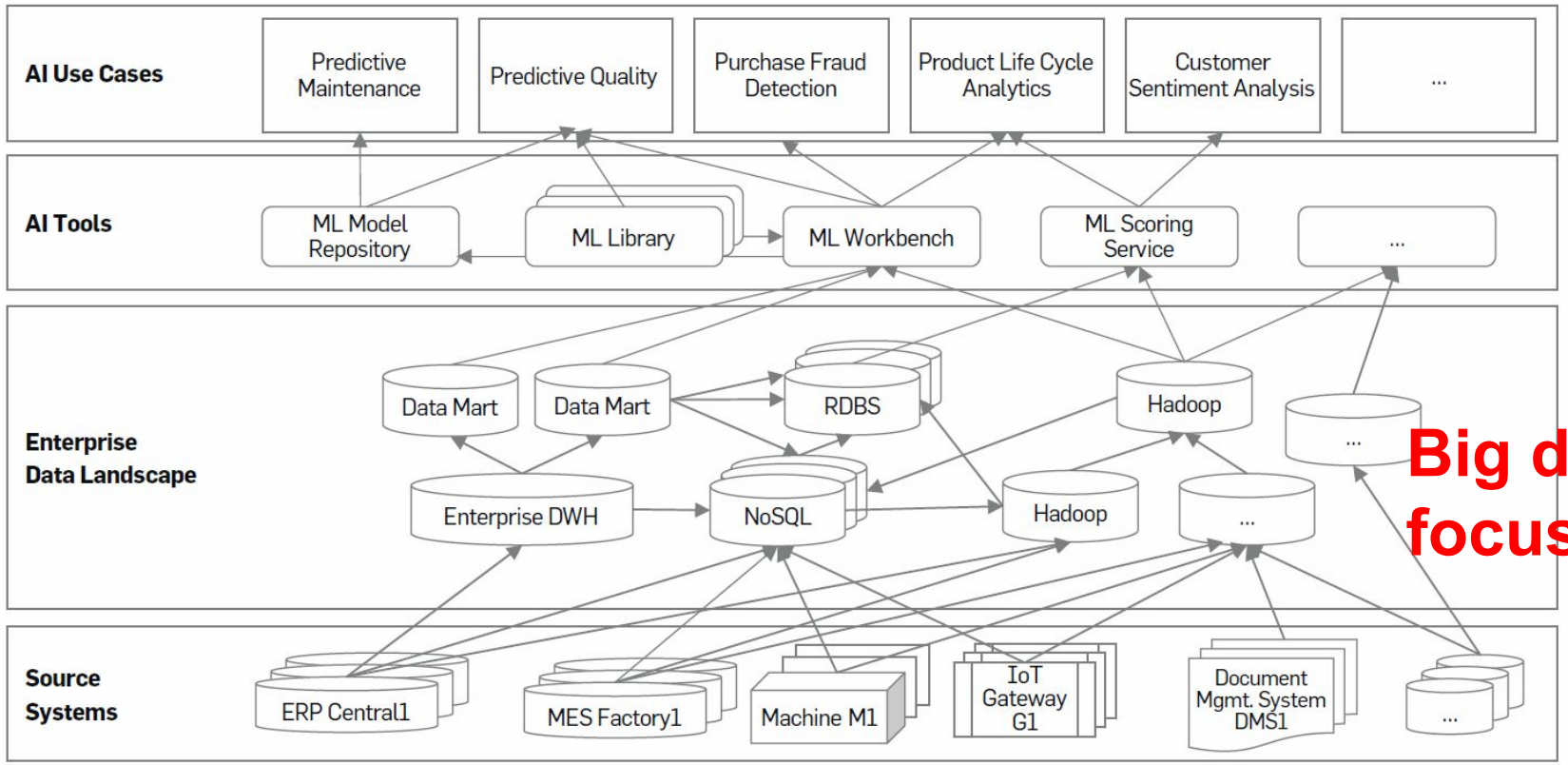
Example AI function integrated into BigQuery





Using techniques/methods studied in Big Data Platforms for GenAI/LLMs

“No AI Without Data”: before LLMs



DWH: Data Warehouse, ERP: Enterprise Ressource Planning, MES: Manufacturing Execution System, ML: Machine Learning, RDBS: Relational Database System

Big data platforms focus

Figure source: There Is No AI Without Data, by Christoph Gröger, Communications of the ACM, November 2021, Vol. 64 No. 11, Pages 98-108, 0.1145/3448247

Big data platforms and ML

It is easy to see the role of big data components we study in this figure

Example: Michelangelo ML from Uber

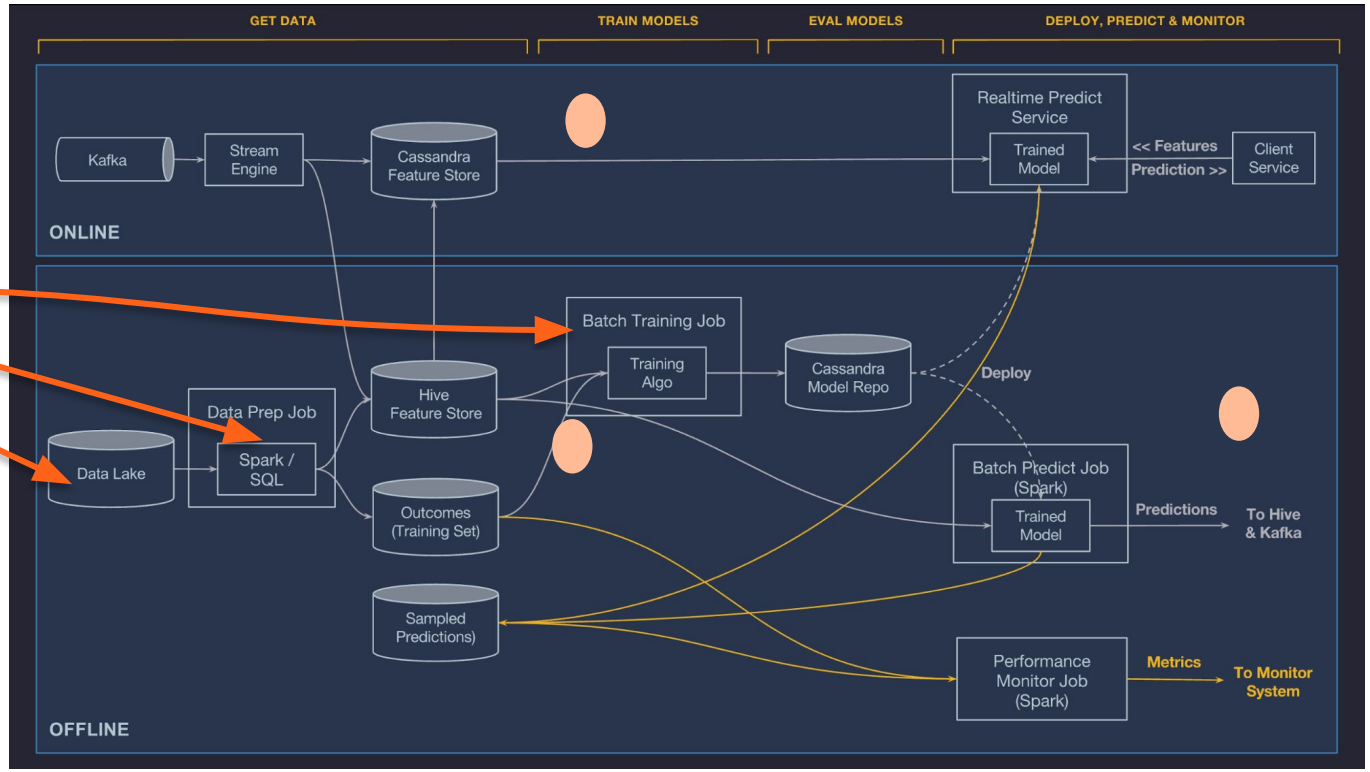


Figure source: https://eng.uber.com/michelangelo-machine-learning-platform/

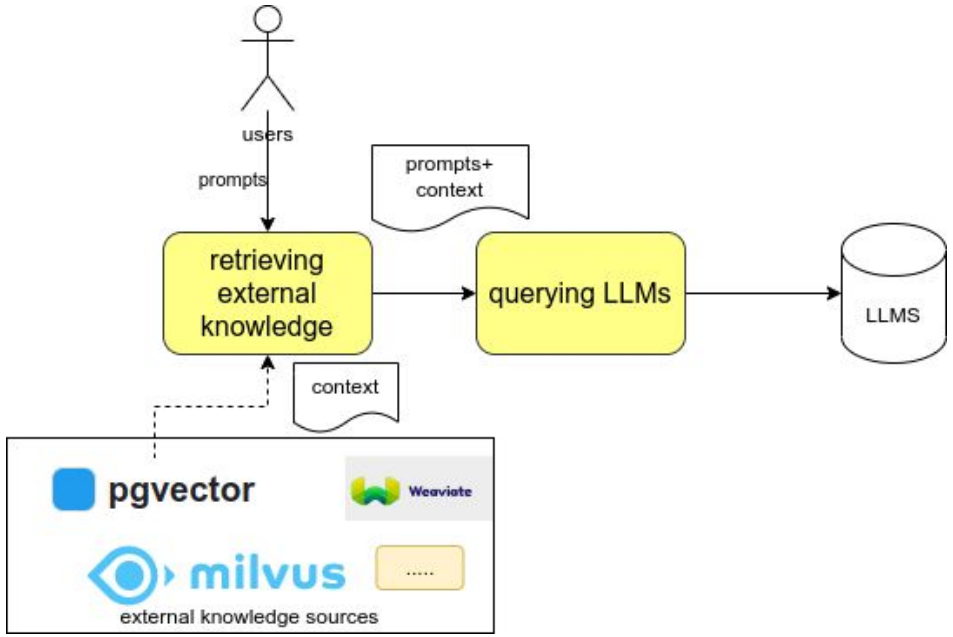
Techniques and methods in big data platforms help to build GenAI/LLMs

- Data management and governance for GenAI/LLMs
  - enable sharing, retrieving and managing big data for training and inference
- Techniques and methods for GenAI/LLMs development
  - data connectors, transformation, data pipelines, and data governance

Support LLMs and Agents: data pipeline for RAG (1)

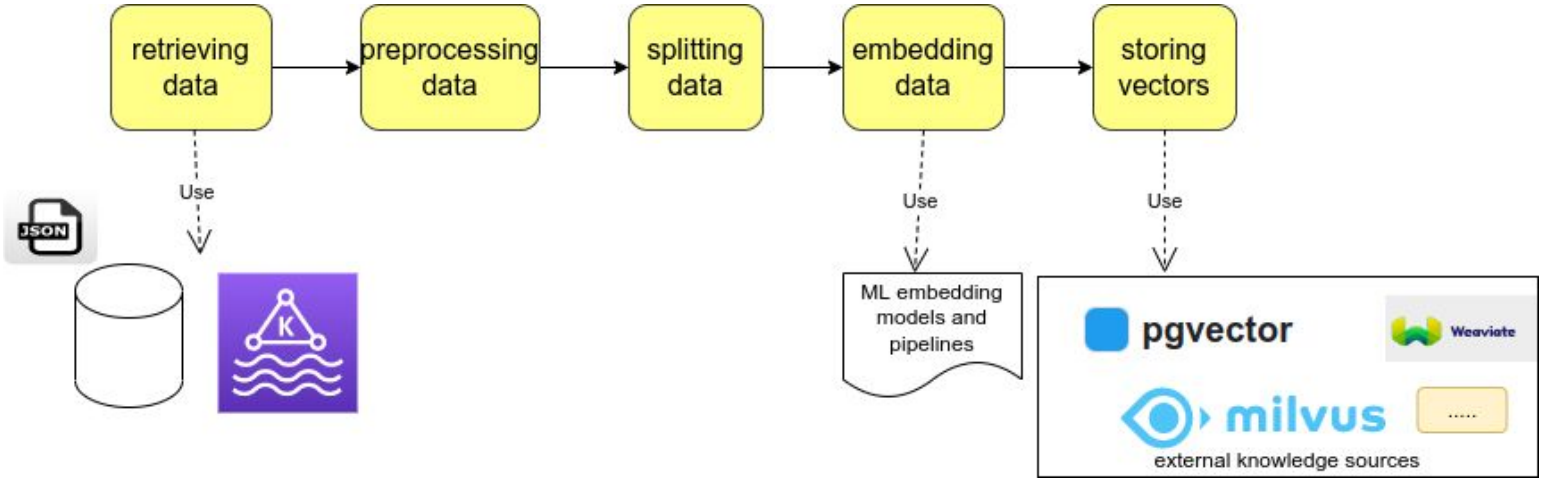
- RAG: Retrieval-Augmented Generation
  - getting facts/relevant accurate knowledge from an external source for LLMs
  - providing context for improving LLM inferences
- Data platforms enable
  - key components for pipelines and databases

RAG for LLMs

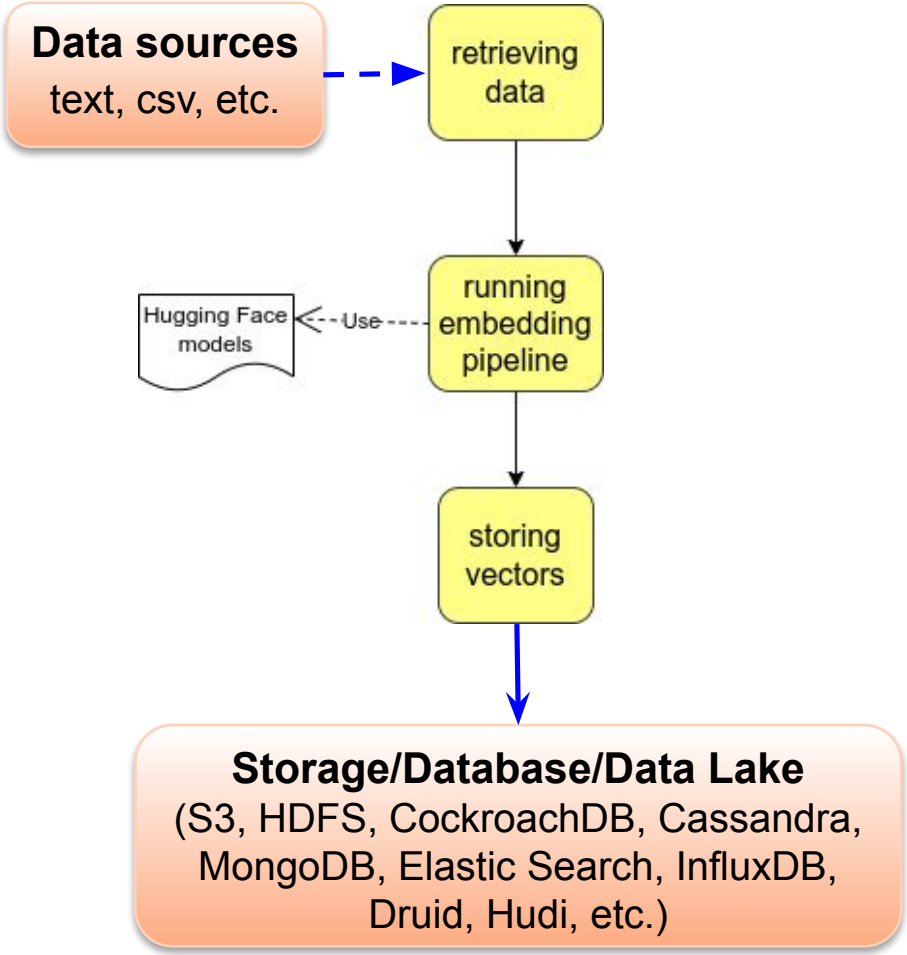


Support LLMs and Agents: data pipeline for RAG (2)

- The process to build RAG is a "big data" pipeline
  - getting data and determining vectors as indexes for data
  - storing data and indexes into databases
  - using common vector databases → **data services**
    - Postgres with pgvector, ChromaDB, Milvus, Weaviate, …



An example with LlamaIndex




```python
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.core.embeddings import resolve_embed_model

#assume the data is under "data" (e.g., csv and text file)
documents = SimpleDirectoryReader("data").load_data()

#specify embedding model for extracting vectors
Settings.embed_model = resolve_embed_model("local:BAAI/bge-small-en-v1.5")

#configure LLM, set None as it illustrates the RAG
Settings.llm = None
# specify the embedding pipeline/task
index = VectorStoreIndex.from_documents(
    documents,
)
#Now we can query Vector to get a context
retriever = index.as_retriever()
query=input("Enter your query:")
contexts = retriever.retrieve(query.strip())
# show context, assume that it context=NodeWithScore with node=TextNode
for context in contexts:
    print(context.node.text)
# also query LLM (but none in this code)
query_engine = index.as_query_engine()
result = query_engine.query(query.strip())
print(result)
```

AI Agents for data platforms

- Emerging, fast evolving development of AI Agents working with data platforms
  - no generic solutions for all: agent functionality is designed for specific data platforms
- Different types of AI Agents in data platforms
  - Agent for managing platform states, workflows, etc. and analyzing root cause performance problems and failures, etc.
  - Agent connectors for data integration
    - get the data but not via low-level protocols
    - embedded AI-based decision for when, where and how to get data
  - Stream processing and agents
    - event-based stream data can be fed into AI agents for reasoning
    - expected throughput and latency as in stream processing

Moving from pre-defined, static ingestion and exchange pipelines & constraints:
AI Agents can enable dynamic and autonomous ingestion & exchange

Examples of current development

AGENT BRICKS

Production AI Agents

Source: https://www.databricks.com/product/artificial-intelligence/agent-bricks

Flink

v0.2.0

Search

What is Flink Agents?

Apache Flink Agents is a brand-new sub-project from the **event-driven streaming agents**.

Building on Flink's battle-tested streaming engine, Apache F **processing and mature state management**, and adds firs **language models (LLMs)**, **prompts**, **tools**, **memory**, **dyn**

Source: https://nightlies.apache.org/flink/flink-agents-docs-release-0.2/docs/get-started/overview/

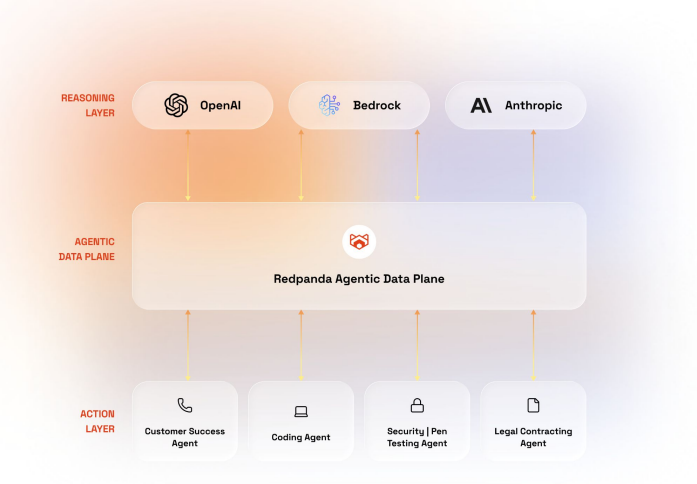
How to work with agent connectors

To get started with agent connectors, follow one of the quick start tutorials.

All agent connectors

Connector	Links
Airtable	Overview | Auth | Reference | Changelog
Amazon Ads	Overview | Auth | Reference | Changelog
Amazon Seller Partner	Overview | Auth | Reference | Changelog

Source: https://docs.airbyte.com/ai-agents/connectors/



Source: https://www.redpanda.com/blog/agentic-data-plane-adp

Summary

- Complex design and engineering issues in big data platforms
  - many big systems, very complex designs and requirements
  - require us to continuously upskill
- New GenAI/LLMs can help simplifying certain tasks
  - GenAI/LLMs and AI Agents are services and paradigms that can be considered as fundamental elements for design and integration
- Expertises from Big Data Platforms are highly important and relevant
  - integrate GenAI/LLM services for applications and scenarios
  - build GenAI/LLMs capabilities for solving big data problems

Thanks!

Hong-Linh Truong
Department of Computer Science

rdsea.github.io

A!

—

Kiitos
aalto.fi